DECCAN EDUCATION SOCIETY'S

KIRTI M. DOONGURSEE COLLEGE (AUTONOMOUS)

DADAR WEST, MUMBAI-400028.

ASSIGNMENT

NAME:- Chauhan Chandan

ROLL NO.:- 17 **CLASS:-** FY Bsc. CS **DIV:-**

SUBJECT Principal Of operating System

PAPER NO. **SEM.** 2 **Year:-**

Date:- 23/02/2024.

Student's Signature

Mark Obtained

___ / 20

Name of Examiner

Signature

Q1 Outline requirements should be satisfied to resolve the critical section problem.

Ans To resolve the critical section problem, several requirements must be satisfied. The critical section problem pertains to the coordination of concurrent processes accessing shared resources. The requirements to resolve this problem typically include

1. Mutual Exclusion:
   - only one process can access a critical section at a time.
   - Ensures that conflicting operations do not occur simultaneously.

2. Progress:
   - If no process is executing in its critical section and some processes wish to their critical sections, then only those processes that are not executing in their remainder section can participate in deciding which will enter next.
   - No process should be prevented indefinitely from entering its critical section.

3. Bounded Waiting:
   - There exists a limit on the number of times other processes can enter their critical sections after a processes has made a request to enter its critical section and above that request is granted.

9167976646 SUPER superst2017@gmail.com

Q.2. Round Robin Scheduling behaves diffently depending on its time quantum. Can the time quantum be set to make round robin behave the same as any of the following algorithm? If so, how?
i) First come - first served ii) Shortest job first.

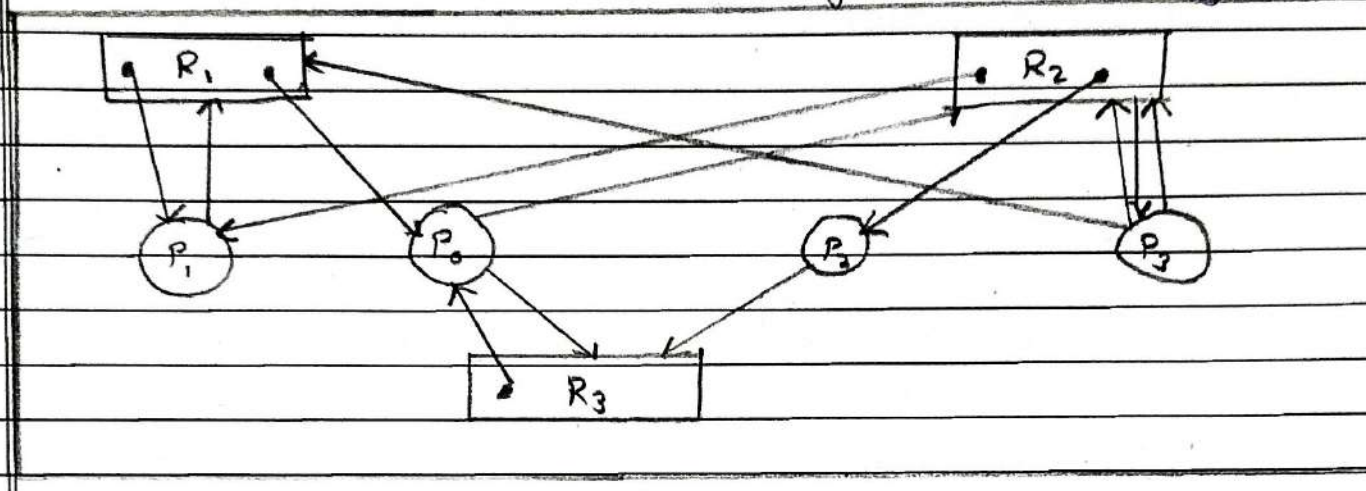
Ans i) First come - first served

You can set the time quantum to be larger than the total burst time of any process in the queue. This effectively eliminates preemption, ensuring that each process runs to completion before the next one starts, mimicking FCFS behavior. In other words the time quantum should be set to infinity or a value larger than the sum of burst times of all processes in the queue.

ii) Shortest job first

you can set the time quantum to be smaller than the burst time of the shortest job in the queue. This ensures that sorter jobs complete quick and are less likely to be preempted by longe jobs By allowing shorter jobs to execute first and minimizing their interruption, you mimic the behaviour of SJF, which selects the shortest job for execution next therefore, setting the time quantum smaller than the burst time of the shortest job prioritizes shorter jobs and achives behavior akin to SJF in the short term.

Q.3 Consider the resource allocation graph in the figure.



i] Find if the System is in deadlock state.
ii] If not find a Safe Sequence

Ans To determine if the system is in a deadlock state let's first calculate the available resource
Here's the allocation & future Need matrix:

Allocation

	r_1	r_2	r_3
P_0	1	0	1
P_1	1	1	0
P_2	0	1	0
P_3	0	1	0

Future Need.

	r_1	r_2	r_3
P_0	0	1	1
P_1	1	0	0
P_2	0	0	1
P_3	1	2	0

Total = (2 3 2)
Allocated = (2 3 1)
Available = Total - Allocation = (0 0 1)

P_2 is need (0 0 1) can be met

And it releases its held resources after running to completion

$A = (0\ 0\ 1) + (0\ 1\ 0) = (0\ 1\ 1)$

P_0 is need $(0\ 1\ 1)$ can be met and release

$A = (0\ 1\ 1) + (1\ 0\ 1) = (1\ 1\ 2)$

P_1 is needs can be met $(1\ 0\ 0)$ and it release

$A = (1\ 1\ 2) + (1\ 1\ 0) = (2\ 2\ 2)$

P_3's need can be met
So, the safe sequences will be
$P_2 - P_0 - P_1 - P_3$

Q.4] Consider the following set of processes with the length of the CPU burst given in miliseconds:

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The process are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0

i] Draw four Gantt charts that illustrate the execution of these processes using the following scheduling algorithm: FCFS and Priority

Ans i] FCFS:

P_1	P_2	P_3	P_4	P_5

0 10 11 13 14 19

ii] Priority:

ii] Compute the average waiting time and average turnaround time for the given scenario.

Priority	Process	Burst time	waiting time	turnaround time
1	P_2	1	0	1
2	P_5	5	1	6
3	P_3	2	6	8
3	P_1	10	8	18
4	P_4	1	18	19

9167976849 SUPER supersc2017@gmail.com

* Average waiting time = $\frac{\text{Sum of waiting time for individual process}}{\text{no. of process}}$

$$= \frac{0 + 1 + 6 + 8 + 18}{5}$$

$$= \frac{33}{5} = 6.60$$

Average waiting time = 6.60

* Average turnaround time = $\frac{\text{Sum of turnaround time for individual process}}{\text{no. of process.}}$

$$= \frac{1 + 6 + 8 + 18 + 19}{5} = \frac{52}{5}$$

Average turnaround time = 10.4

iii] Compare the outcomes of the algorithm and Suggest the best suited one for the given Scenerario

Ans Sceneario : Processes :- P_1, P_2, P_3, P_4, P_5

Arrival times : $P_1(0)$, $P_2(0)$, $P_3(0)$, $P_4(0)$, $P_5(0)$

Burst time : $P_1(10)$, $P_2(1)$, $P_3(2)$, $P_4(1)$, $P_5(5)$

FCFS outcomes:
$P_1 = 10$
$P_2 = 1$
$P_3 = 2$
$P_4 = 1$
$P_5 = 5$

Priority outcomes:
$P_2 = 1$
$P_4 = 1$
$P_3 = 2$
$P_5 = 5$
$P_1 = 10$

9167976646
supers2017@gmail.com

Comparison:

- FCFS schedule Process based solely on their arrival order, executing them in the order they arrive.

- priority (SJF) Schedules processes based on their burst time prioritizing shorter jobs first.

Suggestion:

- Based on the camparison, SJF is better suited for this sceneario as it minimize the average waiting time and ensures a more efficient utilization of resources

- SJF is particularly effective when there is a mix of short and long processes as it prioritizes shorter jobs leading to reduced waiting times and improved system throughout.

Therefor, SJF would be the best suited scheduling algorithm for the given sceneario.

9167976646
SUPER
supersr2017@gmail.com

Q.5] consider the following a System

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

Answer the following questions using the banker's Algorithm:

a] what is the content of the matrix need?

b] Is the System in a safe ~~no~~ state?

c] if a request from process P_1 arrives for (0, 0, 1) can the request be granted immediately?

a] Ans Need matrix :-

Need = Max - Allocation.

Process	Need		
	A	B	C
P_0	0	-1	0
P_1	0	0	2
P_2	-3	0	-3
P_3	-1	-1	-1
P_4	0	0	0

The need matrix is negative, it indicator that the process has already been allocated

b] Safe state or not :

Available ≤ need

P_0 : 000 ≤ 0 -1 0

This is false

∴ process do not execute.

P_1 : 0 00 ≤ 0 02

This is ~~false~~ True

∴ process execute.

New allocation = Available + Allocation

= 000 + 200

= 200

P_2 : 2 0 0 ≤ -3 0 -3

This is false

∴ process do not execute

P_3 : 200 ≤ -1 -1 -1

This is false

∴ process do not execute

P_4 : 200 ≤ 000

This is false.

∴ process do not execute

Starting Available = 020 , No process can complete because all processes have atleast one resource they still need.

Therefore, the System is not in a safe state.

c] i] request ≤ Need

P_1 : 0 0 1 ≤ 0 0 2

This condition is true.

∴ The process P_1 execute

ii] Since process can complete because all process have atleast one resource they still need, the system is not in a safe state after granting the request from process P_1 for (0 0 1)

a] • Available = Available − Request

= 0 0 0 − 0 0 1 = 0 0 1

b] Allocation [P_1] = Allocation [P_1] + Request

= 2 0 0 + 0 0 1 = 2 0 1

c] Need [P_1] = Need [P_1] − Request

= 0 0 2 − 0 0 1 = 0 0 1

3] but there is not in a safe state there fore the request cannot be granted immediately